FALL ALERT

SDSU | San Diego State University

Course: Computer Networks and Distributed Systems

Group: 4

Student 1: Xinyi Yewu 135752523

xyewu4460@sdsu.edu

Student 2: Carmen Chapado Bellido 135758217

cchapadobellid9414@sdsu.edu

Student 3 (Group leader): Diego Perea González 135758256
dpereagonzalez5374@sdsu.edu

Student 4: Nicolás Rodríguez Tabernero 135751977

nrodrigueztabe7129@sdsu.edu

Date: 09/27/2026

SDSU

INDEX

1. Project Overview.....3

2. Project Management Plan.....3

3. Project Deliveries.....5

4. Current Progress.....5

1. Project Overview

FallAlert is a mobile fall detection and alert system. When the smartphone's accelerometer registers a sudden impact followed by a period of inactivity, it triggers an alert through a Python WebSocket relay server. The server routes the event, along with the phone's GPS coordinates at the moment of the fall, to every device currently connected as a receiver.

Unlike fixed home sensors (which suffer from coverage gaps and treat ordinary movement as an alarm), a smartphone-based approach provides location-independent monitoring while the page is open on the phone, delivered through the phone's browser without requiring a native app install.

The main networking challenge is ensuring message reliability over an unreliable multi-hop path. Standard WebSocket runs over TCP, which only guarantees reliable delivery within a single TCP connection, not across the two connections an alert must traverse (phone-to-server, server-to-receiver). If the relay server crashes right after receiving an alert or if a receiver's connection silently drops, the client's send() call still reports success. To solve this, FallAlert implements an application-layer ACK protocol over WebSocket. Each alert carries a sequence number and retry logic (up to three retransmissions upon timeout), providing end-to-end delivery confirmation up to the retry limit.

A secondary challenge is false alarms. Requiring an impact followed by stillness reduces them compared with sensors that react to any movement, but everyday actions like sitting down hard on a couch could still trigger the alert by mistake. Making the detection less sensitive would help, but it could also miss a real fall. To balance this, we plan to add a short countdown on the sender's screen so the user can cancel a false alarm before it is sent, and to test the system during normal activities like walking, running, and climbing stairs.

2. Project Management Plan

Division of work: Diego Perea González: relay server and HTTPS deployment. Nicolás Rodríguez Tabernero: ACK/retransmission layer and gremlin mode. Carmen Chapado Bellido: accelerometer detection (spike + stillness) and GPS capture. Xinyi Yewu: sender/receiver UI, latency logging and experiment graphs. Although the work is divided this way, all four members will help with every part of the project and keep track of its progress.

Period	What's due	Status
Week 7 Oct 5–9	**End-to-end prototype and ALERT protocol** 1. Build the relay server so it forwards incoming messages to every other connected client. 2. Build the sender page with a "simulate fall" button.	**Pending**

	3. Build the receiver page that plays a sound and sends back an ACK. 4. Lock the final ALERT message format (type, id, timestamp) in a short text file in the repo.	
Week 8 Oct 12–16	**Real-device testing and ACK/retry layer** 1. Tunnel the server through ngrok so phones outside the local network can connect. 2. Run repeated trials and log round-trip time in a shared spreadsheet. 3. Compare latency on two networks: home wifi vs. mobile data. 4. Fill in the measured RTT (median of about 20–30 trials). 5. Sequence numbers per alert, 3-second timeout, up to 3 retries before giving up. 6. Receiver skips duplicate alarms but still resends the ACK.	**Pending**
Week 9 Oct 19–23	**Retransmission testing and Gremlin mode** 1. Test the complete alert. 2. Set the timeout just above the worst RTT measured so far, not a guess. 3. Add a server switch that randomly drops alerts/ACKs at a configurable probability. 4. Use it to demo retransmission live in class. 5. Add the post-spike stillness check so running doesn't trigger a false alarm.	**Pending**
Week 10 Oct 26–30	**Multiple receivers and false-alarm cancel** 1. One alert can go to several receivers; it counts as delivered as soon as any one responds. 2. Add a 10-second countdown on the sender screen to cancel a false alarm before it goes out.	**Pending**

Week 11 Nov 2–6	**Loss-and-retry experiment** 1. Send 200 alerts at each loss probability (0.1, 0.3, 0.5) with a max of 3 retries. 2. Compare loss % and average sends per alert against the Lecture 8 formulas. 3. Measure latency on home wifi, campus wifi, and mobile data.	**Pending**
Week 12 Nov 9–13	**Detection accuracy and failure handling** 1. Test fall detection while walking, running, climbing stairs, and sitting down hard; log false positives. 2. Test what happens if the server restarts mid-alert or a receiver drops connection. 3. Fix and document any reliability problems found during testing.	**Pending**
Week 13 Nov 16–20	**Freeze and final test pass** 1. Freeze the code (no more feature changes). 2. Run a full test with the actual phones used in the final presentation. 3. Verify the complete detection, alert, ACK, retry, duplicate-handling, and receiver logic. 4. Record the definitive backup video.	**Pending**
Week 14 Nov 23–25	**Final deliverables and rehearsal** 1. Prepare the final slides, the YouTube video, and the one-page summary. 2. Run at least two timed rehearsals before the Week 15 presentation.	**Pending**

3. Project Deliveries

- **Source code (GitHub repository):** accelerometer-based fall detection, the ACK protocol with retries and duplicate discarding, and support for multiple simultaneous receivers**.**
    - server.py (WebSocket relay)
    - sender.html (sender page)
    - receiver.html (receiver page)
- **Experimental results:** data from the loss-and-retry experiment, plus the false-positive and server-failure tests.
- **Final project report (PDF).**
- **Demo video:** the backup video recorded in Week 13 once the code is frozen (serves as a fallback in case the live demo fails).

4. Current Progress

Period	What's due	Status
Sep 22–28	1. Reviewed existing fall-detection projects, reports and technical blogs, which showed the most prior work relies on bed-based or home-installed sensors that only cover one location and can trigger false alarms. 2. Based on the reviews, we defined the system design: a browser-based sender page that detects a fall (impact spike followed by stillness), a Python WebSocket relay server, and receiver pages that reply with an ACK. 3. Prepared the project plan and division of work for the next 8 weeks. 4. Created and shared a GitHub repository with the whole team. 5. Python installed and verified on every machine. 6. Repository cloned locally so everyone works from the same code.	**Done**

Sep 27	1. Submit Group4_Progress_Report_0928.pdf to Canvas.	**Done**
Sep 28	1. In-class progress presentation (5 minutes + 2 minutes of questions).	**Pending**